



Bases de Datos

Clase: Sentencias Manejo de Datos

Susana Medina Gordillo
susana.medina@correounivalle.edu.co

Continuación lenguaje SQL

Estudiantes

Atributos

codEst	nombre	apellido	edad	ciudad
0011	Juan	Dominguez	17	Cali
0012	Mario	Rodriguez	21	Yumbo
0013	David	Santos	20	Cali
0014	Ximena	Caicedo	18	Cali
0015	Maria	Perez	16	Cali
0016	Felipe	Holguín	23	Yumbo
0017	Juliana	Fernandez	17	Palmira
0018	Andrés	Perez	19	Jamundí

Tuplas

The diagram illustrates the structure of a database table. The word 'Atributos' (Attributes) is positioned above the table, with four arrows pointing to the column headers: 'codEst', 'nombre', 'apellido', 'edad', and 'ciudad'. The word 'Tuplas' (Tuples) is positioned to the left of the table, with eight arrows pointing to each of the data rows, representing individual student records.

Ejemplo: Base de datos de una empresa

CLIENTES (codcli, nombre, direccion, codpostal, codciu)

VENDEDORES (codven, nombre, direccion, codpostal, codpciu, codjefe)

CIUDADES (codciu, nombre, coddep)

DEPARTAMENTOS (coddep, nombre)

ARTICULOS (codart, descrip, precio, stock, stock_min, dto)

FACTURAS (codfac, fecha, codcli, codven, iva, dto)

LINEAS_FAC (codfac, linea, cant, codart, precio, dto)

Restricciones de Integridad



CLIENTES codciu → **CIUDADES**: No acepta nulos.

VENDEDORES codciu → **CIUDADES**: No acepta nulos.

VENDEDORES codjefe → **VENDEDORES** : Acepta nulos.

CIUDADES coddep → **DEPARTAMENTOS**: Acepta nulos.

FACTURAS codcli → **CLIENTES** : Acepta nulos.

FACTURAS codven → **VENDEDORES** : Acepta nulos.

LINEAS_FAC codfac → **FACTURAS** : No acepta nulos.

LINEAS_FAC codart → **ARTICULOS** : No acepta nulos.

Tipos de datos (BD en general)



VARCHAR(n)/ STRING: cadena con un máximo de n caracteres (letras, símbolos o números).

NUMERIC(n,m)/INTEGER/REAL: Número con n dígitos con m números después del punto decimal.

DATE: fechas formadas por día, mes y año. Para incluir la hora también se usa el tipo **TIMESTAMP**.

BOOLEAN: atributos con dos valores: verdadero (**TRUE**) o falso (**FALSE**).

Sentencias de Definición de Datos

Actualización y Eliminación de datos

Sentencia UPDATE / ALTER TABLE

La sentencia UPDATE sirve para **actualizar, borrar o modificar** columnas de una tabla de la base de datos.

Se usa de la siguiente manera:

```
ALTER TABLE nombre_tabla  
ADD nombre_columna tipoDeDato;
```


Ejemplo 1 ALTER TABLE



```
ALTER TABLE Clientes  
ADD Email VARCHAR(200);
```

Ejemplo 2 ALTER TABLE



```
ALTER TABLE facturas  
ADD dto=0  
WHERE dto IS NOT NULL;
```

Ejemplo ALTER TABLE - DROP

```
ALTER TABLE Clientes  
DROP COLUMN codPostal;
```

Algunas bases de datos no permiten
borrar columnas de las tablas!

Sentencia DELETE



La sentencia DELETE sirve para **eliminar** tuplas de una tabla.

Se usa de la siguiente manera:

```
DELETE FROM nombre_tabla  
WHERE condición
```

Ejemplo 1 DELETE



```
DELETE FROM facturas  
WHERE codcli= 133
```

Sentencias de Manejo de Datos:

INSERT

SELECT

Sentencia INSERT



La sentencia **INSERT** tiene como propósito insertar datos en una tabla específica. Para ello la tabla debe **existir**.

Los datos que se van a insertar en la sentencia **INSERT** deben colocarse en el **orden** en el cual fueron **especificados los atributos** (columnas) cuando se creó la tabla.

Sentencia INSERT



```
INSERT INTO nombre_tabla (  
dato_columna1, dato_columna2,  
dato_columna3, dato_columna4,  
dato_columna5, ... dato_columnaN  
);
```


Sentencia INSERT



```
INSERT INTO facturas (6600, '30/04/2007', 111,  
55, 0, NULL);
```

```
INSERT INTO lineas_fac (6600, 1, 4,  
'L76425', 3.16, 25 );
```

```
INSERT INTO lineas_fac (6600, 2, 5,  
'B14017', 2.44, 25 );
```

```
INSERT INTO lineas_fac (6600, 3, 7,  
'L92117', 4.39, 25 );
```

Sentencia SELECT



La sentencia SELECT permite la **consulta** de los datos insertados en una tabla. Se usa de la siguiente manera:

```
SELECT *  
FROM nombre_tabla;
```

SELECT → indica que se va a realizar una consulta

* → indica que se van a consultar todas las columnas

FROM → después se coloca el nombre de la tabla que se va a consultar

Estructura de la Sentencia SELECT

La sentencia SELECT se compone de varias cláusulas, algunas son:

```
SELECT [ DISTINCT ] { * | columna [ , columna ] }  
FROM tabla  
[ WHERE condición_de_búsqueda ]  
[ ORDER BY columna [ ASC | DESC ]  
[ , columna [ ASC | DESC ] ] ;
```

Las cláusulas se ejecutan en un **orden determinado** que debe tenerse en cuenta al momento de hacer una consulta usando SELECT.

Cláusulas de la Sentencia SELECT - Parte I



FROM: especifica la tabla sobre la que se va a realizar la consulta.

WHERE: si sólo se debe mostrar un subconjunto de las filas de la tabla, aquí se especifica la condición que deben cumplir las filas a mostrar; esta condición será un **predicado booleano** con comparaciones unidas por **AND/OR**.

Cláusulas de la Sentencia SELECT - Parte II



SELECT: aquí se especifican las columnas a mostrar en el resultado; para mostrar todas las columnas se utiliza *.

DISTINCT: es un modificador que se utiliza tras la cláusula SELECT para que no se muestren filas repetidas en el resultado (esto puede ocurrir sólo cuando en la cláusula SELECT se prescinde de la clave primaria de la tabla o de parte de ella, si es compuesta).

Cláusulas de la Sentencia SELECT - Parte III



ORDER BY: se utiliza para ordenar el resultado de la consulta.

La cláusula ORDER BY, si se incluye, es siempre la última en la sentencia SELECT. La ordenación puede ser **ascendente** o **descendente** y puede basarse en una sola columna o en varias.

Ejemplo 1



```
SELECT *  
FROM clientes  
;
```

Tuplas Tabla Cliente



codcli	nombre	direccion	codpostal	codciu
001	Juan Perez	calle 2 # 3-1	760501	2
002	Mariano Andrade	cra 1 # 2-1	760501	2
003	Martha Suarez	calle 23 # 5BN-6	760044	1
004	Liliana Bolivar	calle 14 # 7N-9	760030	1
005	Sonia Carvajal	cra 105 # 12-8	760020	1
006	Andrés Peña	cra 100 # 30-48	763531	3

Ejemplo 2



```
SELECT *  
FROM clientes  
ORDER BY codpostal DESC;
```

Tuplas Tabla Cliente



codcli	nombre	direccion	codpostal	codciu
006	Andrés Peña	cra 100 # 30-48	763531	3
001	Juan Perez	calle 2 # 3-1	760501	2
002	Mariano Andrade	cra 1 # 2-1	760501	2
003	Martha Suarez	calle 23 # 5BN-6	760044	1
004	Liliana Bolivar	calle 14 # 7N-9	760030	1
005	Sonia Carvajal	cra 105 # 12-8	760020	1

Ejemplo 3



```
SELECT *  
FROM clientes  
ORDER BY codpostal ASC;
```

Tuplas Tabla Clientes

codcli	nombre	direccion	codpostal	codciu
005	Sonia Carvajal	cra 105 # 12-8	760020	1
004	Liliana Bolivar	calle 14 # 7N-9	760030	1
003	Martha Suarez	calle 23 # 5BN-6	760044	1
001	Juan Perez	calle 2 # 3-1	760501	2
002	Mariano Andrade	cra 1 # 2-1	760501	2
006	Andrés Peña	cra 100 # 30-48	763531	3

Ejemplo 4



```
SELECT *  
FROM clientes  
ORDER BY nombre ASC,  
codpostal  
;
```

Ejemplo 5



```
SELECT *  
FROM clientes  
ORDER BY nombre ASC,  
codpostal ASC  
;
```

Tuplas Tabla Clientes



codcli	nombre	direccion	codpostal	codciu
006	Andrés Peña	cra 100 # 30-48	763531	3
001	Juan Perez	calle 2 # 3-1	760501	2
004	Liliana Bolivar	calle 14 # 7N-9	760030	1
002	Mariano Andrade	cra 1 # 2-1	760501	2
003	Martha Suarez	calle 23 # 5BN-6	760044	1
005	Sonia Carvajal	cra 105 # 12-8	760020	1

Ejemplo 6



```
SELECT *  
FROM clientes  
WHERE codciu=1;
```


Resultado Consulta Tabla Clientes

codcli	nombre	direccion	codpostal	codciu
003	Martha Suarez	calle 23 # 5BN-6	760044	1
004	Liliana Bolivar	calle 14 # 7N-9	760030	1
005	Sonia Carvajal	cra 105 # 12-8	760020	1

Ejemplo 7



```
SELECT *  
FROM clientes  
WHERE codciu=2 AND  
codpostal="760501";
```

Resultado Consulta Tabla Clientes



codcli	nombre	direccion	codpostal	codciu
001	Juan Perez	calle 2 # 3-1	760501	2
002	Mariano Andrade	cra 1 # 2-1	760501	2

Ejemplo 8



```
SELECT codciu,codpostal  
FROM clientes  
WHERE codciu=2 AND  
codpostal="760501";
```

Tuplas Tabla Cliente

codpostal	codciu
760501	2
760501	2

Ejemplo 9



```
SELECT DISTINCT codciu,codpostal  
FROM clientes  
WHERE codciu=2 AND  
codpostal="760501";
```

Tuplas Tabla Cliente



codpostal	codciu
760501	2

Ejercicio en clase

Crear sentencias **SELECT** para responder las siguientes consultas:

- Estudiantes menores de edad
- Estudiantes menores de edad (Popayán)
- Estudiantes con nombre “Andrea”
- Nombres y apellidos de todos los estudiantes.
- Nombres de todos los estudiantes.

Receso 10 min



Funciones y Operadores

Ejercicio: Hotel



El gerente desea conocer:

El promedio de edad de los clientes del hotel.

Consulta



SELECT ?

FROM ?

Consulta



SELECT ?
FROM Clientes

Consulta



```
SELECT edad  
FROM Clientes
```



Operadores Lógicos



AND: Y. Sirve para unir dos o más condiciones que sólo serán verdaderos si se cumplen todas las condiciones, es decir, si son verdaderas.

OR: O. Sirve para unir dos o más condiciones que sólo serán falsas si las dos son falsas.

NOT: Negación. Sirve para negar una expresión lógica.

Operadores de Comparación I



< menor que

> mayor que

<= menor o igual que

>= mayor o igual que

= igual que

!= distinto de

Operadores de Comparación II

a **BETWEEN** x **AND** y: a está entre x y y

a **NOT BETWEEN** x **AND** y: a NO está entre x y y

a **IS NULL**: devuelve **TRUE** si a es nulo

a **IS NOT NULL**: devuelve **TRUE** si a no es un nulo

a **IN** (v1,v2,v3,..., vn): equivale a preguntar si a es igual a alguno de los valores entre paréntesis: **a=v1 OR a=v2 OR..**

Operadores Matemáticos



+ Suma

- Resta

* Multiplicación

/ División

% Módulo / resto de la división entera

^ Potencia ($3^2=9$)

Funciones Matemáticas



ABS(x): Valor absoluto de x

SQRT(x): raíz cuadrada de x

ROUND(x,n,o): redondea x a n dígitos, si n es número positivo. o es de operation y es opcional. Si es cero redondea el valor al número n , si es otro valor diferente de cero trunca el resultado al valor del número n .

Funciones de Columna



COUNT(*): cuenta filas

COUNT(columna): cuenta valores no nulos

SUM(columna): suma los valores de la columna

MAX(columna): valor máximo de la columna

MIN(columna): valor mínimo de la columna

AVG(columna): media aritmética de los valores de la columna

Búsqueda en cadenas (texto)

—

Comparación de Cadenas



Cuando se necesita información sobre las filas que tengan valores similares en una columna determinada se usa **LIKE** en la cláusula **WHERE**.

No se requiere conocer el valor de la columna completa para identificar la fila y escogerla.

WHERE < atributo> **LIKE** modelo

SQL debe proporcionar alguna idea de donde está la cadena de caracteres parcial. Se usan dos caracteres especiales a modo de comodín, que forman parte del modelo. Los caracteres son % (porcentaje) y _ (guión bajo).

LIKE



El (%) se interpreta como un carácter que puede representar cualquier cadena de caracteres de cualquier longitud (hasta de tamaño cero).

Ejemplo: Listar las ciudades que empiecen por **Ca** (Cali, Cartago, Cartagena, ...)

```
SELECT *  
FROM Ciudades  
WHERE nombre LIKE "Ca%";
```

LIKE



El (__) permite comparar en los lugares que aparezca el símbolo _.

Ejemplo: Listar el **nombre** de los empleados cuyo código de supervisor tiene los números **1** en el primer carácter y **3** en el tercer carácter.

```
SELECT nombre
```

```
FROM Empleado
```

```
WHERE codSuperv LIKE '1_3_';
```

Ejemplos

Ejemplo 1 - Hotel



```
SELECT AVG (edad)  
FROM Clientes;
```

Respuesta Consulta 1



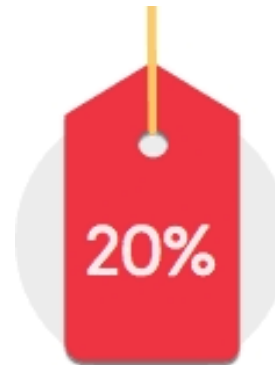
43

Ejemplo 2 - Empresa

```
SELECT  
ROUND (precio * 0.8, 2) AS  
rebajado  
FROM articulos;
```

80%

Pagas el 80% del
precio de la
etiqueta



Descuento del 20%
sobre el precio normal

Respuesta
Consulta 2

Rebajado
38.000
6.000
23.250
15.500
17.450
39.000
62.500
9.200
81.000
10.000
21.000
9.000

Ejemplo 3



```
SELECT  
COUNT (*) FROM articulos  
;
```

Respuesta Consulta 3



100

Ejemplo 4



```
SELECT  
SUM(iva) AS IMPUESTO_VALOR_AGREGADO  
FROM facturas  
;
```


Respuesta Consulta 4



IMPUESTO_VALOR_AGREGADO
128.275.500

Ejemplo 5



```
SELECT  
MAX(valor_dto) AS  
el_mas_barato  
FROM Lineas_Fac  
;
```

Respuesta Consulta 5



<code>el_mas_barato</code>
5.474.000

Ejemplo 6



```
SELECT MIN(iva)  
FROM  
facturas
```

Respuesta Consulta 6



10

Ejemplo 7: DELETE con subconsulta

```
DELETE FROM facturas
WHERE iva= (SELECT MIN(iva)
            FROM facturas )
```

Ejercicio en clase parte 2



Crear sentencias **SELECT** para responder las siguientes consultas:

- Cantidad de estudiantes menores de edad
- Cantidad de estudiantes menores de edad (Cali)
- Promedio de estudiantes menores de edad (Yumbo)
- Nombres y apellidos de todos los estudiantes ordenados decrecientemente según el apellido.
- Nombres y apellidos de todos los estudiantes ordenados ascendentemente según el apellido.
- El estudiante más viejo
- El estudiante más joven